

ОГЛАВЛЕНИЕ

1 ОБЩИЕ СВЕДЕНИЯ	3
1.1 Полное наименование системы и условное обозначение	3
1.2 Назначение документа и основание для разработки	3
1.3 Язык интерфейса и локализация	3
1.4 Краткое описание архитектурного решения	4
2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ СИСТЕМЫ	4
2.1 Назначение системы	4
2.2 Цели создания системы и критерии их достижения	5
2.3 Целевая аудитория и роли пользователей	5
3 УСЛОВИЯ ЭКСПЛУАТАЦИИ И ОГРАНИЧЕНИЯ	6
3.1 Среда выполнения	6
3.2 Сетевое окружение и распределение портов	6
3.3 Ограничения системы	6
4 СОСТАВ И СТРУКТУРА ПРОГРАММНОГО КОМПЛЕКСА	7
4.1 Компонентная структура	7
4.2 Протоколы межсервисного взаимодействия	8
5 ТРЕБОВАНИЯ К ФУНКЦИЯМ СИСТЕМЫ	8
5.1 Требования к функции Ф-01: Регистрация пользователей	9
5.2 Требования к функции Ф-02: Аутентификация и управление JWT-токенами	10
5.3 Требования к функции Ф-03: Управление категориями задач	11
5.4 Требования к функции Ф-04: Управление задачами (Task Management)	12
5.5 Требования к функции Ф-05: Модуль концентрации Pomodoro	13

5.6	Требования к функции Ф-06: Геймификация и статистика продуктивности	14
5.7	Требования к функции Ф-07: Подсистема аудита и логирования (Activity Logger)	15
5.8	Требования к функции Ф-08: Модуль онлайн-оплаты и премиум-доступа	16
5.9	Требования к функциям Ф-09 и Ф-10: Клиентские веб-интерфейсы	16
6	ТРЕБОВАНИЯ К БЕЗОПАСНОСТИ, НАДЕЖНОСТИ И КАЧЕСТВУ	17
6.1	Требования к безопасности	17
6.2	Требования к надежности и обработке ошибок	18
6.3	Требования к производительности	18
7	ТРЕБОВАНИЯ К ВИДАМ ОБЕСПЕЧЕНИЯ	19
7.1	Программное обеспечение	19
7.2	Информационное обеспечение	19
7.3	Техническое обеспечение	20
8	ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ	20
8.1	Виды и стадии приемочных испытаний	20
8.2	Перечень обязательных приемочных проверок	20
8.3	Критерии приемки и фиксация дефектов	22

1 ОБЩИЕ СВЕДЕНИЯ

1.1 Полное наименование системы и условное обозначение

Полное наименование: Программный комплекс управления персональными задачами и продуктивностью «ToDo Notificator».

Краткое наименование / условное обозначение: ПК «ToDo Notificator» (далее – Система).

1.2 Назначение документа и основание для разработки

Настоящий документ представляет собой Техническое задание (ТЗ) на программный комплекс «ToDo Notificator». Документ определяет совокупность функциональных, технических, архитектурных и эксплуатационных требований к Системе, правила взаимодействия её компонентов, а также критерии и порядок приемочного контроля.

Основание для разработки: учебный проект по дисциплине «Тестирование программного обеспечения». Система разрабатывается как объект комплексного функционального, интеграционного и нагрузочного тестирования, а также анализа на наличие логических дефектов и уязвимостей безопасности.

Исходный репозиторий: монорепозитарий проекта *kirill010106/todo-notificator*.

1.3 Язык интерфейса и локализация

Основной язык пользовательского интерфейса клиентских приложений – русский.

Язык системных сообщений серверных интерфейсов прикладного программирования (REST API) и диагностических логов – английский и русский в соответствии со спецификацией протоколов.

1.4 Краткое описание архитектурного решения

Система спроектирована по микросервисной монорепозиторной архитектуре и включает в себя:

- основной сервер прикладной логики (Backend API) на языке Go;
- микросервис аудита и логирования действий пользователей (Activity Logger) на языке Go, использующий протокол gRPC;
- реляционное хранилище учетных записей, задач и сессий на базе СУБД PostgreSQL 16;
- нереляционное документоориентированное хранилище журнала аудита на базе СУБД MongoDB 7;
- два независимых веб-клиента: полнофункциональный SPA-клиент на стеке React 19 / Vite / Ant Design и легковесный статический клиент на стеке Alpine.js / Tailwind CSS;
- изолированную среду контейнеризации и оркестрации на базе Docker Compose.

Почтовый контур (внешний SMTP-шлюз и сервис рассылки) полностью исключен из состава Системы для обеспечения стопроцентной автономности и надежности тестового контура.

2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ СИСТЕМЫ

2.1 Назначение системы

Программный комплекс предназначен для автоматизации процессов индивидуального тайм-менеджмента, ведения списков запланированных задач, фокусировки по технике Pomodoro, непрерывного аудита пользовательской активности и формирования мотивационной аналитики на основе игровых механик (геймификации).

2.2 Цели создания системы и критерии их достижения

Внедрение Системы направлено на решение задач, приведенных в таблице 2.1.

Таблица 2.1 – Целевые показатели создания и внедрения Системы

Цель	Показатель эффективности	Целевое значение
Повышение скорости планирования	Среднее время создания и категоризации задачи	Не более 5 секунд
Обеспечение непрерывной концентрации	Возможность запуска сессий Pomodoro с фиксацией пауз	100% сессий сохраняются в БД
Прозрачность действий пользователя	Фиксация всех критических мутаций (создание, удаление, оплата) в журнале аудита	Не менее 99.9% событий фиксируются в MongoDB
Универсальность доступа	Доступность двух различных реализаций фронтенда (React и Alpine.js)	Одновременная работоспособность на портах 3000 и 3030
Отказоустойчивость ядра	Работоспособность создания задач при недоступности логгера	Задачи создаются без падения HTTP-сервера

2.3 Целевая аудитория и роли пользователей

В Системе определены следующие категории субъектов взаимодействия:

- Гость (неавторизованный пользователь) – имеет доступ к экранным формам аутентификации, регистрации, лендингу возможностей и эндпоинтам проверки состояния сервиса (*/health*);
- Пользователь (авторизованный субъект) – прошедший процедуру регистрации и аутентификации. Обладает изолированным персональным пространством для работы со своими категориями, задачами, таймером Pomodoro и логами активности;
- Премиум-пользователь – субъект с установленным флагом *is_premium = true* (активируется через модуль симуляции онлайн-оплаты YooKassa).

3 УСЛОВИЯ ЭКСПЛУАТАЦИИ И ОГРАНИЧЕНИЯ

3.1 Среда выполнения

Система разворачивается в изолированных контейнерах Docker под управлением Docker Engine версии 24.0 и выше и Docker Compose версии v2.20 и выше.

Клиентские веб-приложения должны поддерживать корректное функционирование в современных браузерах: Google Chrome 120+, Mozilla Firefox 120+, Apple Safari 17+, Microsoft Edge 120+.

Минимальное разрешение экрана для комфортной работы с веб-интерфейсами составляет 360 пикселей по ширине.

3.2 Сетевое окружение и распределение портов

Сетевое взаимодействие сервисов комплекса на локальном хосте разработчика/тестировщика строго регламентировано в таблице 3.1.

Таблица 3.1 – Сетевые интерфейсы и порты программного комплекса

Компонент	Протокол	Внутренний порт	Порт на хосте	URL / Точка доступа
<i>postgres</i>	TCP / PG Wire	5432	5432	<i>localhost:5432</i>
<i>mongo</i>	TCP / Mongo Wire	27017	27017	<i>localhost:27017</i>
<i>activity-logger</i>	gRPC / HTTP/2	50051	50051	<i>localhost:50051</i>
<i>backend</i>	HTTP/1.1 REST	8082	8082	<i>http://localhost:8082</i>
<i>frontend-react</i>	HTTP/1.1 Static	80	3000	<i>http://localhost:3000</i>
<i>frontend-alpine</i>	HTTP/1.1 Static	80	3030	<i>http://localhost:3030</i>

3.3 Ограничения системы

В рамках настоящего ТЗ устанавливаются следующие обязательные проектные ограничения:

- Изоляция данных пользователей: ни при каких обстоятельствах пользователь не должен иметь возможность просматривать, изменять или удалять чужие задачи, категории или логи активности (строгий контроль IDOR);
- Автономность от внешних SMTP-серверов: генерация токенов подтверждения почты и переход по ним осуществляются без задействования реальной доставки электронных писем;
- Stateless аутентификация: авторизация запросов к REST API выполняется исключительно посредством криптографических JWT-токенов в заголовке *Authorization: Bearer <token>*;
- Безопасность секретов: пароли пользователей хранятся в PostgreSQL исключительно в виде криптографических хешей bcrypt. Хранение открытых паролей запрещено.

4 СОСТАВ И СТРУКТУРА ПРОГРАММНОГО КОМПЛЕКСА

4.1 Компонентная структура

В состав Системы входят следующие структурные модули:

- 1) Ядро Backend API (*backend*) – монолитный RESTful сервис на языке Go (роутер Chi, ORM/SQL через чистый SQL драйвер *pgx / database/sql*), обеспечивающий валидацию входных данных, бизнес-логику управления сущностями и выдачу токенов;
- 2) Микросервис аудита (*activity-logger*) – отдельный бинарный сервис на языке Go, реализующий контракт gRPC (*proto/activity_logger/v1/activity_logger.proto*) и осуществляющий запись структурированных документов истории в коллекцию MongoDB;

- 3) Реляционная база данных PostgreSQL – персистентное хранение сущностей: *users*, *tasks*, *categories*, *pomodoro_sessions*, *user_stats*, *refresh_tokens*, *payments*;
- 4) Документная база данных MongoDB – персистентное хранение журнала аудита событий *activity_logs*;
- 5) Клиент React (*frontend-react*) – одностраничное веб-приложение, скомпилированное сборщиком Vite и раздаваемое через веб-сервер Nginx на порту 3000;
- 6) Клиент Alpine (*frontend-alpine*) – легковесный веб-интерфейс на Alpine.js и стилях Tailwind CSS, раздаваемый через веб-сервер Nginx на порту 3030.

4.2 Протоколы межсервисного взаимодействия

Взаимодействие компонентов регламентируется следующими протоколами:

- Клиент <-> Backend API: протокол HTTP/1.1, форматы передачи данных – JSON (*application/json*), авторизация заголовком *Authorization: Bearer <access_token>*;
- Backend API <-> Activity Logger: бинарный RPC протокол gRPC поверх HTTP/2 с сериализацией Google Protocol Buffers v3;
- Backend API <-> PostgreSQL: бинарный протокол PostgreSQL Wire Protocol с пулом постоянных соединений;
- Activity Logger <-> MongoDB: официальный протокол MongoDB Wire Protocol.

5 ТРЕБОВАНИЯ К ФУНКЦИЯМ СИСТЕМЫ

Сводный перечень функциональных требований к программному комплексу приведен в таблице 5.1.

Таблица 5.1 – Перечень функций программного комплекса

Код функции	Наименование функции	Содержание функции	Приоритет
Ф-01	Регистрация пользователей	Создание учетной записи с валидацией email и надежности пароля	Обязательная
Ф-02	Аутентификация и JWT	Выдача пар access/refresh токенов, ротация и логгаут	Обязательная
Ф-03	Управление категориями	Создание, редактирование, просмотр и удаление пользовательских категорий	Обязательная
Ф-04	Управление задачами	CRUD задач, приоритизация, сроки, фильтрация, пагинация	Обязательная
Ф-05	Таймер Pomodoro	Запуск, пауза, остановка интервалов фокуса, учет времени	Обязательная
Ф-06	Геймификация и статистика	Расчет очков опыта (XP), уровней, стриков и учет просрочек	Обязательная
Ф-07	Аудит и логирование (gRPC)	Асинхронная фиксация событий и чтение истории из MongoDB	Обязательная
Ф-08	Модуль онлайн-оплаты	Создание заказа, симуляция вебхука YooKassa, премиум-доступ	Обязательная
Ф-09	Пользовательский SPA React	Компактный интерфейс задач и помодоро на Ant Design	Обязательная
Ф-10	Легковесный клиент Alpine	Расширенный интерфейс с фильтрацией, аудитом и лендингом	Обязательная

5.1 Требования к функции Ф-01: Регистрация пользователей

- 1) Эндпоинт регистрации: *POST /api/v1/register*;
- 2) Входные параметры (JSON).
 - *email* (строка, обязательное) – адрес электронной почты;
 - *password* (строка, обязательное) – пароль пользователя.
- 3) Правила валидации.
 - *email* должен соответствовать корректному синтаксису адреса электронной почты (RFC 5322);
 - *password* должен иметь длину не менее 8 символов и содержать как минимум одну заглавную букву, одну строчную букву, одну цифру и один специальный символ.

4) Поведение системы.

- При успешной валидации пароль хешируется алгоритмом `bcrypt`;
- В таблице `users` создается новая запись;
- В таблице `user_stats` автоматически инициализируется начальная статистика пользователя (`points = 0`, `level = 1`, `current_streak = 0`, `best_streak = 0`);
- В сервис `activity-logger` отправляется событие `USER_REGISTERED`;
- Сервер возвращает HTTP статус `200 OK` (или `201 Created`) с телом `{«status»:«OK», «user_id»: <id>}`.

5) Обработка исключительных ситуаций.

- Если email уже зарегистрирован – возвращается HTTP статус `409 Conflict` с сообщением об ошибке `user already exists`;
- Если формат входных данных некорректен – возвращается HTTP статус `400 Bad Request` с перечнем нарушенных правил валидации.

5.2 Требования к функции Ф-02: Аутентификация и управление JWT-токенами

1) Эндпоинт входа: `POST /api/v1/login`.

- Принимает `email` и `password`;
- При совпадении хеша пароля генерирует пару токенов:
 - `access_token` – JWT токен (алгоритм `HS256`), подписанный секретом `APP_SECRET`, со сроком жизни 15 минут (900 секунд). Содержит клеймы: `uid` (ID пользователя), `email`, `is_premium`, `is_verified`, `exp`, `iat`;
 - `refresh_token` – криптографически стойкая случайная строка длиной 64 символа, сохраняемая в таблице `refresh_tokens` со сроком жизни 30 дней;
- В `activity-logger` отправляется событие `USER_LOGGED_IN`;
- Возвращается HTTP статус `200 OK` с телом `{«status»:«OK», «access_token»: "...", «refresh_token»: "...", «expires_in»: 900}`;

- При неверном логине или пароле возвращается HTTP статус *401 Unauthorized* (или *400 Bad Request*) с сообщением об ошибке.
- 2) Эндпоинт обновления токенов: *POST /api/v1/refresh*.
- Принимает тело *{«refresh_token»: "..."};*
 - Проверяет существование и срок действия токена в таблице *refresh_tokens*;
 - Реализует механизм **ротации**: старый *refresh_token* удаляется (или инвалидируется), генерируется и сохраняется новый *refresh_token*, а также выдается новый *access_token*;
 - При передаче недействительного или истекшего токена возвращается HTTP статус *401 Unauthorized*.
- 3) Эндпоинт выхода из системы: *POST /api/v1/logout*.
- Требуется авторизация через Bearer токен;
 - Удаляет все активные refresh-токены данного пользователя из таблицы *refresh_tokens*;
 - Возвращает HTTP статус *200 OK*.

5.3 Требования к функции Ф-03: Управление категориями задач

- 1) Эндпоинты.
- *GET /api/v1/categories* – получение списка всех категорий авторизованного пользователя;
 - *POST /api/v1/categories* – создание новой категории;
 - *GET /api/v1/categories/{category_id}* – получение подробных сведений о категории;
 - *PATCH /api/v1/categories/{category_id}* – обновление названия, цвета или иконки категории;
 - *DELETE /api/v1/categories/{category_id}* – удаление категории.
- 2) Валидация полей категории.

- *name* (строка, от 1 до 50 символов, не может быть пустым или состоять только из пробелов);
- *color* (строка, HEX-код формата *#RRGGBB* или допустимое системное имя цвета);
- *icon* (строка, идентификатор иконки).

3) Правила целостности и безопасности.

- Пользователь имеет право запрашивать, модифицировать и удалять **только свои собственные** категории. Попытка обращения к чужой категории должна возвращать HTTP статус *403 Forbidden* или *404 Not Found*;
- При удалении категории связанные с ней задачи не удаляются каскадно, а их поле *category_id* сбрасывается в *NULL* (отсутствие категории).

5.4 Требования к функции Ф-04: Управление задачами (Task Management)

1) Эндпоинты.

- *GET /api/v1/tasks* – получение списка задач с поддержкой фильтрации;
- *POST /api/v1/tasks* – создание новой задачи;
- *PATCH /api/v1/tasks/{task_id}* – частичное обновление параметров задачи;
- *DELETE /api/v1/tasks/{task_id}* – удаление задачи.

2) Модель данных задачи.

- *title* (строка, обязательное, длина от 1 до 255 символов);
- *description* (строка, опциональное, максимальная длина 2000 символов);
- *category_id* (целое число, опциональное, должно принадлежать текущему пользователю);
- *priority* (перечисление: *low*, *medium*, *high*, по умолчанию *medium*);
- *deadline* (дата и время ISO 8601, опциональное);

- *reminder_at* (дата и время ISO 8601, опциональное);
- *status* (перечисление: *pending* – в ожидании, *done* – выполнено, *burnt* – просрочено).

3) Поведение системы при смене статуса.

- При переходе задачи в статус *done* пользователю в *user_stats* автоматически начисляется **+10 очков опыта (points)**;
- Если задача не выполнена и текущее время превысило *deadline*, статус задачи переводится в *burnt*, а в статистике инкрементируется *total_burnt_tasks*;
- Каждое создание, изменение статуса и удаление задачи инициирует отправку gRPC события в *activity-logger* (*TASK_CREATED*, *TASK_UPDATED*, *TASK_DELETED*).

4) Параметры фильтрации эндпоинта *GET /api/v1/tasks*.

- *status* (*pending*, *done*, *burnt*);
- *category_id* (целое число);
- *priority* (*low*, *medium*, *high*);
- *limit* (целое число, по умолчанию 50, максимум 100);
- *offset* (целое число, по умолчанию 0).

5.5 Требования к функции Ф-05: Модуль концентрации Pomodoro

1) Эндпоинты.

- *POST /api/v1/pomodoros/start* – старт новой сессии фокуса;
- *GET /api/v1/pomodoros/active* – получение текущей незавершенной сессии;
- *POST /api/v1/pomodoros/{id}/pause* – постановка активной сессии на паузу / снятие с паузы;
- *POST /api/v1/pomodoros/{id}/stop* – досрочная остановка либо успешное завершение сессии.

2) Правила функционирования таймера.

- Пользователь не может запустить вторую сессию Pomodoro, если у него уже есть сессия в статусе *active* или *paused*. Сервер должен возвращать ошибку валидации или HTTP *409 Conflict*;
- Длительность сессии по умолчанию составляет 25 минут (настраиваемый диапазон от 5 до 60 минут);
- Сессия может быть привязана к конкретной задаче (*task_id*) либо запущена независимо (*task_id = null*);
- При успешном завершении сессии в таблице *pomodoro_sessions* фиксируется *completed_at*, а в *user_stats* увеличивается счетчик *total_pomodoros*.

5.6 Требования к функции Ф-06: Геймификация и статистика продуктивности

1) Эндпоинты.

- *GET /api/v1/me/stats* – получение метрик текущего пользователя;
- *PATCH /api/v1/me/stats* – ручная корректировка/обновление показателей.

2) Состав статистических метрик (*user_stats*).

- *points* – накопленные баллы опыта (XP);
- *level* – текущий уровень пользователя (целое число от 1 до 100). Уровень вычисляется по математической формуле:

$$\text{\$level} = \lfloor \sqrt{\text{points} / 100} \rfloor + 1$$
- *total_pomodoros* – общее число успешно завершенных помидоров фокуса;
- *total_burnt_tasks* – количество невыполненных задач с истекшим дедлайном;
- *current_streak* – непрерывная серия дней, в которые была закрыта хотя бы одна задача;
- *best_streak* – исторический рекорд непрерывной серии дней.

3) Правило расчета стриков.

- Если пользователь закрыл задачу в день SD , и последняя активность была в день $SD-1$, *current_streak* увеличивается на 1;
- Если последняя активность была ранее чем $SD-1$, *current_streak* сбрасывается в 1;
- Если *current_streak* > *best_streak*, значение *best_streak* обновляется.

5.7 Требования к функции Ф-07: Подсистема аудита и логирования (Activity Logger)

- 1) Контракт микросервиса регламентирован файлом *proto/activity_logger/v1/activity_logger.proto*;
- 2) Состав сообщения *LogRequest*.
 - *user_id* (int64) – идентификатор пользователя;
 - *action* (string) – тип события (*USER_REGISTERED*, *USER_LOGGED_IN*, *TASK_CREATED*, *TASK_UPDATED*, *TASK_DELETED*, *POMODORO_STARTED*, *POMODORO_COMPLETED*, *PAYMENT_SUCCESS*);
 - *entity_id* (int64) – ID связанной сущности (ID задачи, сессии и т.д.);
 - *details_json* (string) – метаданные события в виде JSON-строки.
- 3) Правила обработки.
 - Activity Logger сохраняет документ в MongoDB со структурой: *_id*, *user_id*, *action*, *entity_id*, *details_json*, *timestamp* (Unix time в миллисекундах);
 - Клиентский вызов метода *GET /api/v1/me/logs* возвращает отсортированный по убыванию времени список действий с поддержкой pagination (*limit*, *offset*);
 - При недоступности MongoDB микросервис возвращает клиенту gRPC ошибку с кодом *Internal*, однако основной Backend API не должен прерывать обработку пользовательского HTTP-запроса (отказоустойчивое логирование).

5.8 Требования к функции Ф-08: Модуль онлайн-оплаты и премиум-доступа

1) Эндпоинты.

- *POST /api/v1/payments/create* – инициализация платежа на оформление Premium-подписки;
- *POST /api/v1/webhooks/yookassa* – прием входящего вебхука об изменении статуса платежа.

2) Поведение.

- При создании платежа формируется уникальный идентификатор транзакции и возвращается ссылка на подтверждение оплаты (в локальном тестовом стенде генерируется mock-ссылка);
- При поступлении вебхука со статусом *payment.succeeded* Система находит соответствующую запись в таблице *payments*, переводит её в статус *succeeded*, обновляет поле *is_premium = true* в таблице *users* и генерирует событие аудита *PAYMENT_SUCCESS*.

5.9 Требования к функциям Ф-09 и Ф-10: Клиентские веб-интерфейсы

1) Клиент React 19 (*frontend-react*, порт 3000).

- Реализация на функциональных компонентах React с использованием хуков Zustand и библиотеки Ant Design;
- Компонент авторизации с валидацией пароля на клиенте в реальном времени;
- Панель категорий с возможностью выбора цветового маркера;
- Список задач с быстрым чекбоксом выполнения, редактированием и вызовом модального окна таймера Pomodoro.

2) Клиент Alpine.js (*frontend-alpine*, порт 3030).

- Легковесная клиентская архитектура на Alpine.js и утилитарных стилях Tailwind CSS;

- Наличие промо-лендинга с описанием возможностей продукта;
- Расширенная фильтрация списка задач по временным горизонтам: «Все», «Сегодня», «Предстоящие», «Просроченные», «Выполненные»;
- Интерактивная вкладка просмотра журнала аудита (Logs) с возможностью динамической дозагрузки записей по кнопке «Загрузить еще»;
- Визуальный дашборд геймификации с отображением стрика, прогресс-бара уровня и эффектами анимации.

6 ТРЕБОВАНИЯ К БЕЗОПАСНОСТИ, НАДЕЖНОСТИ И КАЧЕСТВУ

6.1 Требования к безопасности

- 1) Контроль доступа к данным (IDOR Protection).
 - Все манипуляции с сущностями (задачи, категории, сессии помодоро, логи) должны производиться со строгой фильтрацией по *user_id*, извлеченному из проверенного JWT-токена;
 - Запрещена передача *user_id* в теле клиентских запросов для идентификации владельца.
- 2) Криптографическая защита.
 - Секретный ключ подписи JWT (*APP_SECRET*) должен иметь длину не менее 32 символов и передаваться через переменные окружения;
 - Пароли пользователей должны хешироваться с использованием соли и фактора трудоемкости *bcrypt* не менее 10.
- 3) Межсайтовые запросы (CORS).
 - Сервер REST API должен явно объявлять разрешенные источники (*Allowed Origins*), включая *http://localhost:3000*, *http://localhost:3030* и *http://localhost:5173*.
- 4) Защита от XSS и инъекций.

- Все параметры SQL-запросов должны передаваться исключительно через параметризованные плейсхолдеры (*\$1*, *\$2*), конкатенация строк в SQL запрещена;
- Пользовательский ввод в текстовых полях задач и категорий должен экранироваться перед выводом в DOM-дерево.

6.2 Требования к надежности и обработке ошибок

1) Единый формат ошибок API.

- Все ошибочные ответы должны возвращаться с соответствующим HTTP-кодом (400, 401, 403, 404, 409, 500) и унифицированной JSON-структурой.

```
json
{
  «status»: «Error»,
  «error»: «Описание возникшей ошибки»
}
```

- Не допускается передача клиенту необработанных внутренних системных stack trace или сырых ошибок драйвера СУБД.

2) Отказоустойчивость.

- Сбои в микросервисе *activity-logger* или СУБД MongoDB не должны приводить к падению основного процесса Backend API или невозможности создать/отредактировать задачу.

6.3 Требования к производительности

1) Время отклика REST API.

- 95-й перцентиль времени отклика серверной части на типовые операции чтения и записи задач при нагрузке до 50 rps не должен превышать 200 мс.

2) Размер статических ресурсов.

- Общий объем бандла клиентских приложений при первой загрузке (без учета кэша) не должен превышать 2 МБ.

7 ТРЕБОВАНИЯ К ВИДАМ ОБЕСПЕЧЕНИЯ

7.1 Программное обеспечение

Состав используемых базовых программных средств приведен в таблице 7.1.

Таблица 7.1 – Программное обеспечение компонентов

Компонент	Стек технологий / Версия	Назначение
Backend API	Go 1.25, Chi router, pgx	Основная серверная бизнес-логика
Activity Logger	Go 1.25, gRPC, Protobuf v3, mongo-go-driver	Сервис аудита действий
СУБД PostgreSQL	PostgreSQL 16 Alpine	Реляционная база данных
СУБД MongoDB	MongoDB 7 Community	Документная база данных аудита
Frontend React	Node.js 20, React 19, Vite, Ant Design	Современный SPA-клиент
Frontend Alpine	Nginx Alpine, Alpine.js 3, Tailwind CSS	Легковесный клиент
Инструмент миграций	Goose	Автоматическое применение DDL-миграций
Контейнеризация	Docker Engine, Docker Compose v2	Изолированное развертывание

7.2 Информационное обеспечение

Информационное обеспечение включает:

- Реляционную схему данных PostgreSQL, определенную набором версионированных миграций в директории *backend/migrations*;
- Схему документов MongoDB в коллекции *activity_logs*;
- Протокольные спецификации Protobuf в директории *proto/activity_logger/v1/*.

7.3 Техническое обеспечение

Для запуска стенда требуется вычислительная машина со следующими характеристиками:

- Архитектура x86-64 (AMD64) или ARM64;
- Не менее 2 физических ядер процессора;
- Не менее 4 ГБ оперативной памяти (RAM);
- Не менее 5 ГБ свободного дискового пространства;
- Сетевой стек TCP/IP с поддержкой loopback-интерфейса *localhost*.

8 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ

8.1 Виды и стадии приемочных испытаний

Приемка программного комплекса осуществляется в форме функционального и интеграционного тестирования по утвержденной Программе и методике испытаний (ПМИ).

Испытания проводятся на локальном автономном стенде, развернутом с помощью команды:

Листинг 8.1 – Команда запуска стенда для приемочных испытаний

```
docker compose -f docker-compose.local.yml up -d --build
```

8.2 Перечень обязательных приемочных проверок

Система считается успешно прошедшей приемочные испытания, если в полном объеме выполнены следующие проверки:

1) Проверка инициализации и развертывания стенда:.

- Все 6 контейнеров (*todo-postgres-local*, *todo-mongo-local*, *todo-activity-logger-local*, *todo-backend-local*, *todo-frontend-react-local*, *todo-frontend-alpine-local*) успешно переходят в состояние *Up / healthy*;
- Эндпоинт *GET http://localhost:8082/api/v1/health* возвращает статус *200 OK* и тело *{«status»:«OK»}*;

- Веб-клиенты отвечают кодом *200 OK* по адресам *http://localhost:3000* и *http://localhost:3030*.

2) Проверка аутентификации и авторизации (Ф-01, Ф-02):.

- Регистрация нового пользователя с корректными email и надежным паролем завершается успешно;
- Повторная регистрация с тем же email отклоняется с кодом *409 Conflict*;
- Вход по валидным учетным данным возвращает валидный JWT *access_token* и *refresh_token*;
- Запрос к защищенному эндпоинту (например, *GET /api/v1/tasks*) без заголовка авторизации отклоняется с кодом *401 Unauthorized*;
- Вызов эндпоинта *POST /api/v1/refresh* с валидным refresh-токеном осуществляет ротацию и возвращает новую валидную пару токенов.

3) Проверка управления категориями (Ф-03):.

- Создание категории с корректным названием и HEX-цветом сохраняется в БД;
- Попытка создания категории с пустым именем отклоняется с кодом *400 Bad Request*;
- Пользователь А не может получить доступ к категориям пользователя Б (проверка изоляции).

4) Проверка жизненного цикла задач (Ф-04):.

- Создание задачи с заполнением всех полей завершается со статусом *201 Created* (или *200 OK*);
- Фильтрация задач по статусу и категории возвращает строго релевантные записи;
- Перевод задачи в статус *done* инкрементирует баллы опыта пользователя (+10 XP);
- Удаление задачи физически удаляет её из выборки *GET /api/v1/tasks*.

5) Проверка модуля Pomodoro (Ф-05):.

- Запуск сессии возвращает активную сессию с корректным временем старта;
- Повторная попытка запуска параллельной сессии без завершения предыдущей блокируется с кодом ошибки;
- Завершение сессии фиксирует общее время и обновляет счетчик *total_pomodoros*.

6) Проверка аудита и надежности (Ф-07, Раздел 6):.

- Действия пользователя (регистрация, создание задачи) фиксируются в коллекции MongoDB и доступны по запросу *GET /api/v1/me/logs*;
- При принудительной остановке контейнера MongoDB (*docker stop todo-mongo-local*) вызовы API на создание задач в PostgreSQL продолжают успешно обрабатывать без падения бэкенда.

7) Проверка клиентских веб-интерфейсов (Ф-09, Ф-10):.

- В React-клиенте на *http://localhost:3000* пользователь может зарегистрироваться, авторизоваться, создать категорию и задачу;
- В Alpine.js клиенте на *http://localhost:3030* отображаются созданные задачи, корректно работают фильтры, журнал логов и дашборд геймификации;
- В консолях браузера при выполнении штатных сценариев отсутствуют необработанные исключения уровня *Error*.

8.3 Критерии приемки и фиксация дефектов

Приемочные испытания считаются успешно завершенными, если все перечисленные в пункте 8.2 сценарии выполняются в точном соответствии с заявленными требованиями ТЗ.

Любое расхождение между фактическим поведением программного комплекса и требованиями разделов 5 и 6 настоящего ТЗ квалифицируется как **дефект (баг)**, подлежащий документированию в баг-репорте тестировщика с указанием шагов воспроизведения, ожидаемого и фактического результатов.